Dataflow Modeling in Verilog HDL

Digital Design and RTL Fundamentals

October 12, 2026

Contents

1 Introduction **3**

2 Basic Syntax **3**
2.1 Example . . . 3

3 Continuous Assignment **4**
3.1 Buffer . . . 4
3.2 Inverter . . . 4
3.3 AND Gate . . . 4

4 Operators Used in Dataflow Modeling **5**
4.1 Bitwise Operators . . . 5
4.2 Logical Operators . . . 5

5 Dataflow Modeling of Basic Logic Gates **5**
5.1 AND Gate . . . 5
5.2 OR Gate . . . 6
5.3 NOT Gate . . . 6
5.4 NAND Gate . . . 6
5.5 NOR Gate . . . 7
5.6 XOR Gate . . . 7
5.7 XNOR Gate . . . 7

6 Truth Table of Basic Logic Gates **8**

7 Dataflow Modeling of a Half Adder **8**
7.1 Verilog Code . . . 8
7.2 Truth Table . . . 8

8 Dataflow Modeling of a Full Adder **9**
8.1 Verilog Code . . . 9

9 Dataflow Modeling of a 2-to-1 Multiplexer **9**
9.1 Using Boolean Operators . . . 9
9.2 Using the Conditional Operator . . . 10

10 Dataflow Modeling of a 4-to-1 Multiplexer **10**

10.1 Boolean Expression 10
10.2 Verilog Code 10
10.3 Selection Table 10

11 Dataflow Modeling of a 2-to-4 Decoder **11**
11.1 Verilog Code 11
11.2 Truth Table 11

12 Dataflow Modeling of a Comparator **11**
12.1 Verilog Code 11

13 Dataflow Modeling of a 4-Bit Adder **12**
13.1 Verilog Code 12

14 Concatenation and Replication **12**
14.1 Concatenation Example 12
14.2 Replication Example 12

15 Delay in Continuous Assignments **13**
15.1 Syntax 13
15.2 Example 13

16 Testbench for Dataflow Modeling **13**
16.1 Half Adder Testbench 13

17 Dataflow Modeling vs. Structural Modeling **14**

18 Dataflow Modeling vs. Behavioral Modeling **14**

19 Advantages of Dataflow Modeling **14**

20 Limitations of Dataflow Modeling **14**

21 Common Errors and Best Practices **15**

22 Dataflow Modeling Summary **15**

23 Summary **16**

1 Introduction

Dataflow modeling is a modeling style in Verilog HDL used to describe relationships between signals using expressions and continuous assignments.

It describes how outputs depend on inputs through logical, relational, and arithmetic expressions.

The primary construct used in dataflow modeling is the `assign` statement.

Dataflow modeling is commonly used for combinational circuits such as logic gates, adders, multiplexers, decoders, and comparators.

The main features are:

- Uses continuous assignment statements.
- Uses Boolean and arithmetic expressions.
- Describes relationships between signals.
- Supports bitwise and logical operators.
- Is suitable for combinational logic.
- Provides concise descriptions of circuits.

2 Basic Syntax

The general syntax of a continuous assignment is:

```
assign output_signal = expression;
```

The keyword `assign` continuously drives the left-hand-side net with the expression on the right-hand side.

Whenever an operand changes, the expression is reevaluated and the driven output is updated.

In Verilog, a continuous assignment normally drives a net such as a `wire`.

2.1 Example

```
module dataflow_example (
    input a,
    input b,
    output y
);

    assign y = a & b;

endmodule
```

The output `y` is the bitwise AND of inputs `a` and `b`.

3 Continuous Assignment

A continuous assignment establishes a relationship between an expression and a net.

3.1 Buffer

```
module buffer_gate (
    input a,
    output y
);

    assign y = a;

endmodule
```

The output follows the input.

3.2 Inverter

```
module inverter (
    input a,
    output y
);

    assign y = ~a;

endmodule
```

The bitwise NOT operator complements the input.

3.3 AND Gate

```
module and_gate (
    input a,
    input b,
    output y
);

    assign y = a & b;

endmodule
```

The output is 1 only when both inputs are 1.

4 Operators Used in Dataflow Modeling

Verilog provides several operators for continuous assignment expressions.

4.1 Bitwise Operators

Operator	Name	Operation
&	Bitwise AND	ANDs corresponding bits.
|	Bitwise OR	ORs corresponding bits.
~	Bitwise NOT	Complements each bit.
^	Bitwise XOR	Produces 1 when corresponding bits differ.
~^	Bitwise XNOR	Produces 1 when corresponding bits are equal.
~&	NAND	AND followed by complement.
~|	NOR	OR followed by complement.

4.2 Logical Operators

Operator	Name	Operation
&&	Logical AND	True when both operands are logically true.
||	Logical OR	True when at least one operand is logically true.
!	Logical NOT	Inverts the logical truth value.

Bitwise operators work on individual bits. Logical operators evaluate logical truth values and produce a one-bit result.

5 Dataflow Modeling of Basic Logic Gates

5.1 AND Gate

Boolean equation:

$$Y = A \cdot B$$

```
module and_gate (
    input A,
    input B,
    output Y
);

    assign Y = A & B;

endmodule
```

5.2 OR Gate

Boolean equation:

$$Y = A + B$$

```
module or_gate (
    input A,
    input B,
    output Y
);

    assign Y = A | B;

endmodule
```

5.3 NOT Gate

Boolean equation:

$$Y = \overline{A}$$

```
module not_gate (
    input A,
    output Y
);

    assign Y = ~A;

endmodule
```

5.4 NAND Gate

Boolean equation:

$$Y = \overline{A \cdot B}$$

```
module nand_gate (
    input A,
    input B,
    output Y
);

    assign Y = ~(A & B);

endmodule
```

5.5 NOR Gate

Boolean equation:

$$Y = \overline{A + B}$$

```
module nor_gate (
    input A,
    input B,
    output Y
);

    assign Y = ~(A | B);

endmodule
```

5.6 XOR Gate

Boolean equation:

$$Y = A \oplus B$$

```
module xor_gate (
    input A,
    input B,
    output Y
);

    assign Y = A ^ B;

endmodule
```

5.7 XNOR Gate

Boolean equation:

$$Y = \overline{A \oplus B}$$

```
module xnor_gate (
    input A,
    input B,
    output Y
);

    assign Y = ~(A ^ B);

endmodule
```

6 Truth Table of Basic Logic Gates

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

7 Dataflow Modeling of a Half Adder

A half adder adds two single-bit inputs and generates sum and carry outputs.

The Boolean equations are:

$$S = A \oplus B$$

$$C = A \cdot B$$

7.1 Verilog Code

```
module half_adder (
    input A,
    input B,
    output S,
    output C
);

    assign S = A ^ B;
    assign C = A & B;

endmodule
```

7.2 Truth Table

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

8 Dataflow Modeling of a Full Adder

A full adder adds three single-bit inputs: A, B, and carry-in Cin.

It produces sum S and carry-out Cout.

The Boolean equations are:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

8.1 Verilog Code

```
module full_adder (
    input A,
    input B,
    input Cin,
    output S,
    output Cout
);

    assign S = A ^ B ^ Cin;
    assign Cout = (A & B) | (A & Cin) | (B & Cin);

endmodule
```

The sum is calculated using XOR operations. The carry output is 1 when at least two inputs are 1.

9 Dataflow Modeling of a 2-to-1 Multiplexer

A multiplexer selects one of two inputs using a select signal.

The Boolean equation is:

$$Y = \overline{S}A + SB$$

9.1 Using Boolean Operators

```
module mux2to1 (
    input A,
    input B,
    input S,
    output Y
);
    assign Y = (~S & A) | (S & B);

endmodule
```

9.2 Using the Conditional Operator

```
module mux2to1_conditional (
    input A,
    input B,
    input S,
    output Y
);

    assign Y = S ? B : A;

endmodule
```

The conditional operator selects B when S is 1 and A when S is 0.

10 Dataflow Modeling of a 4-to-1 Multiplexer

A 4-to-1 multiplexer selects one of four inputs using two select signals.

10.1 Boolean Expression

$$\begin{aligned} Y = &\overline{S_1}\,\overline{S_0}D_0 \\ &+ \overline{S_1}S_0D_1 \\ &+ S_1\overline{S_0}D_2 \\ &+ S_1S_0D_3 \end{aligned}$$

10.2 Verilog Code

```
module mux4to1 (
    input D0,
    input D1,
    input D2,
    input D3,
    input S0,
    input S1,
    output Y
);
    assign Y = S1
             ? (S0 ? D3 : D2)
             : (S0 ? D1 : D0);
endmodule
```

10.3 Selection Table

S1	S0	Output
0	0	D0
0	1	D1
1	0	D2
1	1	D3

11 Dataflow Modeling of a 2-to-4 Decoder

A decoder converts a binary input into a one-hot output pattern.

11.1 Verilog Code

```
module decoder2to4 (
    input A,
    input B,
    output [3:0] Y
);

    assign Y[0] = ~A & ~B;
    assign Y[1] = ~A &  B;
    assign Y[2] =  A & ~B;
    assign Y[3] =  A &  B;

endmodule
```

11.2 Truth Table

A	B	Y[3]	Y[2]	Y[1]	Y[0]
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

12 Dataflow Modeling of a Comparator

A comparator determines whether two binary values are equal or whether one is greater than the other.

12.1 Verilog Code

```
module comparator4 (
    input [3:0] A,
    input [3:0] B,
    output equal,
    output greater,
    output less
);

    assign equal   = (A == B);
    assign greater = (A > B);
    assign less    = (A < B);

endmodule
```

The relational operators compare the unsigned 4-bit values in this example.

13 Dataflow Modeling of a 4-Bit Adder

The addition operator can describe a multi-bit adder concisely.

13.1 Verilog Code

```
module adder4 (
    input [3:0] A,
    input [3:0] B,
    input Cin,
    output [3:0] Sum,
    output Cout
);

    assign {Cout, Sum} = {1'b0, A} + {1'b0, B} + Cin;

endmodule
```

The operands are extended to five bits before addition. The five-bit result is assigned to the concatenated output.

14 Concatenation and Replication

Concatenation combines signals into a vector.

14.1 Concatenation Example

```
module concatenation_example (
    input [3:0] A,
    input [3:0] B,
    output [7:0] Y
);

    assign Y = {A, B};

endmodule
```

The upper four bits of Y contain A, and the lower four bits contain B.

14.2 Replication Example

```
module replication_example (
    input a,
    output [3:0] Y
);
    assign Y = {4{a}};
endmodule
```

If a is 1, the output is 1111. If a is 0, the output is 0000.

15 Delay in Continuous Assignments

Verilog allows a delay to be specified in a continuous assignment.

15.1 Syntax

```
assign #delay output_signal = expression;
```

15.2 Example

```
`timescale 1ns/1ps

module delayed_and (
    input A,
    input B,
    output Y
);

    assign #5 Y = A & B;

endmodule
```

The assignment specifies a delay of 5 ns under the given timescale.

Delay specifications are useful in simulation models. They do not guarantee the physical delay of synthesized hardware.

16 Testbench for Dataflow Modeling

A testbench applies input combinations and observes the outputs of the design.

16.1 Half Adder Testbench

```
module half_adder_tb;
    reg A, B;
    wire S, C;
    half_adder dut (.A(A), .B(B), .S(S), .C(C) );
    initial begin
        $monitor("Time=%0t A=%b B=%b S=%b C=%b", $time, A, B, S, C);
        A = 0; B = 0;
        #10 A = 0; B = 1;
        #10 A = 1; B = 0;
        #10 A = 1; B = 1;
        #10 $finish;
    end
endmodule
```

Compile the half-adder module together with the testbench before running the simulation.

17 Dataflow Modeling vs. Structural Modeling

Feature	**Dataflow Modeling**	**Structural Modeling**
Description	Uses expressions and continuous assignments.	Uses gates and module connections.
Main construct	`assign`.	Gate and module instantiations.
Representation	Describes signal relationships.	Describes component interconnections.
Typical use	Boolean expressions and arithmetic.	Gate-level and hierarchical design.

18 Dataflow Modeling vs. Behavioral Modeling

Feature	**Dataflow Modeling**	**Behavioral Modeling**
Description	Signal relationships through expressions.	Procedural or algorithmic behavior.
Main construct	Continuous `assign`.	`always` and `initial`.
Typical use	Combinational logic.	Combinational and sequential logic.

19 Advantages of Dataflow Modeling

- Provides concise descriptions of Boolean functions.
- Is suitable for combinational circuits.
- Supports arithmetic and logical expressions.
- Simplifies adders, multiplexers, and comparators.
- Allows synthesis tools to infer hardware from supported expressions.
- Makes Boolean equations easy to modify.

20 Limitations of Dataflow Modeling

- Large expressions may become difficult to understand and maintain.
- Complex control logic may be easier to describe behaviorally.
- Continuous assignments do not by themselves describe clocked state updates.
- Incorrect widths or signedness can produce unexpected arithmetic results.
- Multiple conflicting drivers can cause unknown values during simulation.
- Simulation delays do not guarantee physical implementation timing.

21 Common Errors and Best Practices

Common errors include:

- Using undeclared signals.
- Assigning continuous expressions to inappropriate variable types in Verilog.
- Ignoring operator precedence.
- Ignoring vector widths and signedness.
- Confusing bitwise and logical operators.
- Creating conflicting drivers on a net.

Recommended practices include:

- Use parentheses in complex expressions.
- Declare signal widths explicitly.
- Use meaningful signal names.
- Check signed and unsigned arithmetic.
- Use continuous assignments for combinational relationships.
- Verify designs with testbenches.
- Prefer parameters for configurable widths.

22 Dataflow Modeling Summary

Construct	**Purpose**
`assign`	Creates a continuous assignment.
`&`	Bitwise AND operation.
`|`	Bitwise OR operation.
`~`	Bitwise complement.
`^`	Bitwise XOR operation.
`?:`	Conditional operator.
`{A,B}`	Concatenates signals into a vector.
`{N{A}}`	Replicates a signal or expression.
`==`	Equality comparison.
`>` and `<`	Greater-than and less-than comparisons.

23 Summary

- Dataflow modeling describes circuits using expressions and continuous assignments.
- The `assign` statement is the main construct used in dataflow modeling.
- Bitwise operators describe logic functions.
- Arithmetic operators describe adders and other arithmetic circuits.
- The conditional operator is useful for multiplexers.
- Concatenation and replication simplify vector assignments.
- Dataflow modeling is widely used for combinational RTL design.

Key takeaway: Dataflow modeling describes relationships between signals using continuous assignments and expressions. It provides a concise way to represent combinational logic and arithmetic circuits in Verilog HDL.